

Compilador fase 1: Análise léxica e sintática

O objetivo desse trabalho é implementar as fases de análise léxica e sintática de um compilador para linguagem **Portugol**, baseada na **Linguagem Pascal** com palavras reservadas em português.

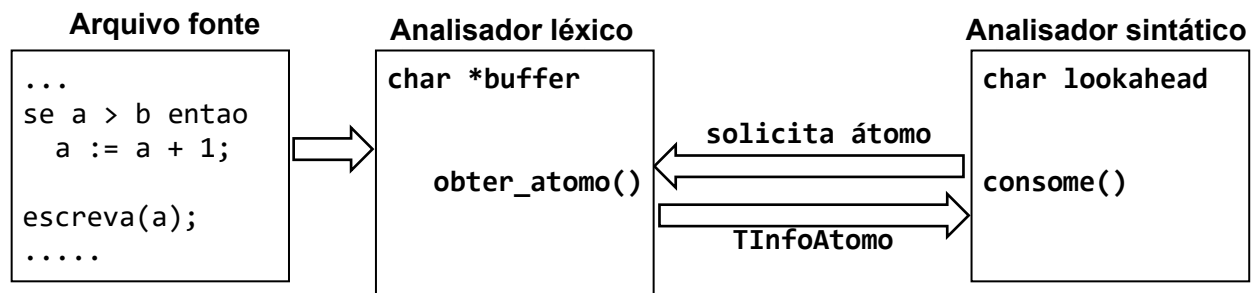


Figura 1: Interação entre Analisador Léxico e Sintático

Na sua implementação o **analisador léxico deve atender as demandas do analisador sintático**, conforme ilustrado na Figura 1. Por conta disso você deve implementar, **OBRIGATORIAMENTE**, as funções **consome()** (**analisador sintático**) que realizará chamadas da função **obter_atomo()** (**analisador léxico**) que retornará uma estrutura **TInfoAtomo**. A seguir as declarações das funções e da estrutura **TInfoAtomo**:

```
TInfoAtomo obter_atomo(); // implementado no analisador léxico
void consome(TAtomo atomo); // implementado no analisador sintático
```

```
typedef struct{
    TAtomo atomo;
    int linha;
    union{
        int numero; // atributo do átomo constint (constante inteira)
        char id[16]; // atributo identificador
        char ch; // atributo do átomo constchar (constante caractere)
    }atributo;
}TInfoAtomo;
```

Na sequência são apresentadas a gramática e as especificações léxicas da linguagem da linguagem **Portugol**, que devem ser seguidas rigorosamente.

Gramática da linguagem Portugol

A sintaxe da linguagem **Portugol** é descrita por uma gramática na notação **EBNF**, vale ressaltar que a notação **EBNF** utiliza os símbolos especiais **[, {, }, [,], (,)** na sua metalinguagem¹. Os **<não-terminais>** da gramática são nomes entre parênteses angulares **< e >** e os símbolos **terminais** (átomos do analisador léxico) estão em **negrito** ou entre apóstrofos (Ex: **';**'), observe que os símbolos especiais da notação **EBNF** estão em vermelho (ex: **{, }**) e os terminais em apóstrofo (ex: **'{'** e **'}'**). A construção **{ α }** denotará a repetição da cadeia **α** zero, uma ou mais vezes (**α***) e **[β]** é equivalente a **β|λ**, ou seja, indica que a cadeia **β** é opcional. Considere que o símbolo inicial da gramática é **<programa>**:

```
<programa> ::= algoritmo identificador ';' <bloco> '.'
<bloco> ::= <declaração_variáveis> <declaração_rotinas> <comando_composto>
<declaração_variáveis> ::= [ var <lista_variaveis> ';' { <lista_variaveis> ';' } ]
<lista_variaveis> ::= identificador { ',' identificador } ':' <tipo>
<declaracao_de_rotinas> ::= { <declaração_de_função> | <declaração_de_procedimento> }
```

¹ Uma **metalinguagem** é uma linguagem usada para descrever a gramática, usando símbolos e regras de produção.

```

<declaração_de_função> ::= funcao <tipo> identificador <parametros_formais>
                             <declaracao_de_variaveis> <comando_composto>

<declaracao_de_procedimento> ::= procedimento identificador <parametros_formais>
                                   <declaracao_de_variaveis> <comando_composto>

<tipo> ::= caractere | inteiro | logico

<parâmetros_formais> ::= '(' <parâmetro_formal> { ; parâmetro_formal } ')' | '(' ')'

<parâmetro_formal> ::= [var] <lista_variaveis>

<comando_composto> ::= inicio <comando> { ';' <comando> } fim

<comando> ::=
    <comando_atribuição>      |
    <comando_entrada>         |
    <comando_saida>           |
    <comando_condicional>     |
    <comando_repeticao>        |
    <chamada_procedimento>    |
    <comando_composto>

<comando_atribuição> ::= identificador '=' <expressão>

<comando_entrada> ::= leia '(' identificador { ',' identificador } ')'

<comando_saida> ::= escreva '(' <lista_expressão> ')'

<comando_condicional> ::= se <expression> entao <statement> [ senao <statement> ]

<comando_repeticao> ::= enquanto <expression> faca <statement>

<chamada_procedimento> ::= identificador [ '(' <lista_expressão> ')' ]

<lista_expressão> ::= <expressão> { ',' <expressão> }

<expressão> ::= <expressão_simples> [ <operador_relacional> <expressão_simples> ]

<operador_relacional> ::= '<' | '<' | '<=' | '>=' | '>' | '=' |

<expressão_simples> ::= <termo> { <operador_adição> <termo> }

<operador_adição> ::= '+' | '-' | mod | ou

<termo> ::= <fator> { <operador_multiplicação> <fator> }

<operador_multiplicação> ::= '*' | div | e

<fator> ::= identificador [ '(' <lista_expressão> ')' ] | constint | constchar |
            '(' <expressão> ')' | ( '+' | '-' | nao ) <fator> | verdadeiro | falso

```

Especificações Léxicas

- **Caracteres Delimitadores:** Os caracteres delimitadores: espaços em branco, quebra de linhas, tabulação e retorno de carro (' ', '\n', '\t', '\r') deverão ser eliminados (ignorados) pelo analisador léxico, mas o controle de linha (contagem de linha) deverá ser mantido.
- **Comentários:** Na linguagem **Portugol** só existe comentários de várias linhas que começam com '{-' e termina com '-}', lembrando que a contagem de linha deve ser mantida dentro do comentário.

Importante: Os comentários devem ser repassados para o analisador sintático para serem processados e descartados.

- **identificador**: Os identificadores começam com uma letra, seguidos de zero ou mais letras (maiúscula ou minúscula) ou *underline* ‘_’ ou dígitos, limitados a 15 caracteres. Caso seja encontrado um identificador com **mais de 15** caracteres deve ser retornado **ERRO** pelo analisador léxico. A seguir a definição regular para o átomo **identificador**.

letra $\rightarrow a|b|\dots|z|A|B|\dots|Z|$

digito $\rightarrow 0|1|\dots|9$

identif $\rightarrow \text{letra}(\text{letra} | _ | \text{digito})^*$

Importante: Na saída do compilador, para átomo **identificador**, **deverá ser impresso o lexema que gerou o átomo**, ou seja, a sequência de caracteres reconhecida. Para isso o lexema deverá ser armazenado na estrutura **TInfoAatomo** no campo **atributo.id**.

- **Palavras reservadas**: As palavras reservadas da linguagem **Portugol** são:
algoritmo | caractere | div | e | enquanto | entao | escreva | faca | falso | fim | função | identificador | inicio | inteiro | leia | logico | mod | ou | procedimento | se | senao | var | verdadeiro

Importante: O compilador **não é sensível ao caso**, ou seja, os identificadores e palavras reservadas podem ser informados com caracteres maiúsculos ou minúsculos, sendo assim, para o lexema "**alGoRiTmo**" ou "**ALGORITMO**" o átomo retornado é **ALGORITMO**.

- **constchar**: para constante caractere são aceitos qualquer caractere da tabela ASCII entre apóstrofo, por exemplo: ‘a’ ou ‘0’.

Importante: O caractere que gerou o átomo **constchar** deve ser armazenado na estrutura **TInfoAatomo** no campo **atributo.ch** pelo analisador léxico e depois impresso na tela na saída do compilador.

- **constint**: Para constante inteira o compilador reconhece somente números inteiros na **notação decimal**, com possibilidade de escrever o número na notação exponencial (potências de 10), conforme expressão regular abaixo:

constint $\rightarrow \text{digito}^+((\text{E}(+|\epsilon)\text{digito}^+)|\epsilon)$

Exemplos de constantes inteiras: 1, 000, 124, 12E2 (=1200), 12E+2 (=1200)

Importante: O analisador léxico deve retornar o valor número que gerou o átomo **constint** utilizando a estrutura **TInfoAatomo** preenchendo o campo **atributo.numero**, que depois será impresso na saída do compilador.

Execução do Compilador

O compilador deve ler o arquivo fonte, com o nome informado por linha de comando, e apresentar na tela do computador, a linha e a descrição de todos os átomos reconhecidos no arquivo fonte, o número de linhas analisadas caso o programa esteja sintaticamente correto.

Abaixo temos um exemplo de arquivo fonte na linguagem **Portugol**, sem erros léxicos ou sintáticos, e sua respectiva saída na tela:

Arquivo fonte de entrada.

1	{-
2	programa le dois numeros inteiros e encontra o maior
3	-}
4	algoritmo ex1;
5	var num_1, NUM_2:inteiro; maior:inteiro; tEste:caractere;
6	inicio
7	leia (Num_1,Num_2);
8	Se Num_1 > Num_2 entao
9	maior := num_1
10	SEnao
11	maior := num_2;
12	Teste := ‘f’; {- so pra testar o char -}
13	escreva (maior) {- imprime o maior valor -}
14	fim.

Saída do compilador na tela

```
# 1:comentario
# 4:algoritmo
# 4:identificador: ex1
# 4:ponto_virgula
# 5:var
# 5:identificador: num_1
# 5:virgula
# 5:identificador: NUM_2
# 5:dois_pontos
# 5:inteiro
# 5:ponto_virgula
# 5:identificador: maior
# 5:dois_pontos
# 5:inteiro
# 5:ponto_virgula
# 5:identificador: tEste
# 5:dois_pontos
# 5:caractere
# 5:ponto_virgula
# 6:inicio
...
14 linhas analisadas, programa sintaticamente correto
```

Caso seja detectado um **erro léxico ou sintático** o compilador deve-se emitir uma mensagem de erro explicativa e terminar a execução do programa. A mensagem explicativa deve informar a linha do erro, o tipo do erro (léxico ou sintático) e caso seja um erro sintático, deve-se informar qual era o **átomo esperado** e qual foi o **átomo encontrado** na análise, veja abaixo um exemplo de saída com erro do compilador.

Arquivo fonte de entrada.

```
1 Algoritmo ex2;
2 Inicio
3     ESCREVA(maior ;
4 Fim.
```

Exemplo de saída do compilador na tela

```
# 1:algoritmo
# 1:identificador: ex2
# 1:ponto_virgula
# 2:inicio
# 3:escreva
# 3:abre_par
# 3:identificador: maior
# 3:erro sintatico, esperado [)] encontrado [;]
```

Observações importantes:

O programa deve ser devidamente documentado e poderá ser desenvolvido em grupo de **até dois alunos**. É **imprescindível** que os nomes dos integrantes do grupo sejam mencionados no início do arquivo fonte do trabalho.

Fica **terminantemente proibido** o uso de ferramentas de **Inteligência Artificial**, como o ChatGPT, para a **geração automática do código do projeto**. Qualquer tentativa de burlar esta restrição será considerada uma infração disciplinar, conforme o **COMUNICADO DA FCI de 05/02/2025**.

Critérios de Avaliação do Trabalho:

1. Funcionamento do programa:

- O trabalho deve ser desenvolvido na **linguagem C** e será testado usando o compilador GCC do instalado na versão Ubuntu-22.04, caso não tenha o Ubuntu instalado consulte o documento **WSL_Windows_Ubuntu_VSCode.pdf** disponível no Moodle.

- O trabalho deve ser desenvolvido na **linguagem C** e será testado usando o compilador **gcc** do Linux, para compilar seu programa com o seguinte comando abaixo, considere que o programa fonte do seu compilador seja `compilador.c`:

```
gcc -Wall -Wno-unused-result -g -Og compilador.c -o compilador
```

Para executar use:

```
./compilador <nome do arquivo de entrada>
```

- Caso o programa não compile ou não execute **será atribuída a nota 0 ao trabalho**.
- Caso programa apresentem **warning** durante a compilação ou não finalize com retorno igual a 0, será descontado 1.0 (um ponto) por **warning** relatado.
- O programa não pode conter erro de acesso memória, caso tenha a nota será reduzida à metade. Para testar a memória do seu programa use o **valgrind** conforme descrito abaixo:

```
valgrind --leak-check=yes ./compilador <nome do arquivo de entrada>
```

- Entregue um arquivo **readme.txt** explicando como compilar e rodar o seu trabalho, informe também até qual parte do trabalho foi concluída, além de relatar quaisquer *bugs* ou erros identificados na sua implementação. No arquivo você pode compartilhar alguma decisão de *design* e implementação que foram tomadas durante o desenvolvimento.

2. Atendimento à especificação do enunciado:

- **O quão fiel é o programa quanto à descrição do enunciado**, o seu programa deve seguir a gramática definida e realizar a leitura de **programa fonte** armazenado em **arquivo** com o nome informado **por linha de comando**.
- Clareza e organização, programas com código confuso (linhas longas, variáveis com nomes não-significativos,) e desorganizado (sem indentação, sem comentários,) também serão penalizados.
- Para esclarecer alguma dúvida quanto a entrega do projeto, poderá ser **requisitado uma entrevista com os integrantes** que implementaram o projeto, com intuito de validar se as regras acima foram todas atendidas.

Considerações sobre uso de ferramentas de IA

As atividades práticas propostas ao longo da disciplina têm como principal objetivo promover o desenvolvimento do raciocínio, da autonomia e da capacidade de resolução de problemas. Quando nós professores desenvolvemos um **projeto prático** ou **exercício**, procuramos criar desafios exijam um conhecimento que você ainda não tem, mas que pode adquirir com um pouco de esforço e estudo.

O que isso quer dizer? O que estamos interessados **não em ouvir a resposta correta**, mas que você **passe por um processo de aprendizado** que o leve a essa resposta.

Quando você consulta uma ferramenta de IA (ou um colega, ou site com soluções), você pula etapas desse processo. Isso tem alguns benefícios:

1. Você termina mais rápido;
2. Exige menos esforço;
3. Você obtém a pontuação pela entrega da atividade;
4. Você tem a sensação de ter concluído a tarefa.

E, acredite, nós entendemos que esses são realmente benefícios, mas, **pensando em aprendizado**, existe um grande **prejuízo**, pois você não passou pela etapa de reflexão e aprendizado que o exercício propunha.

Alguns de vocês podem argumentar que usam a IA só pra explicar o processo para resolver o problema. Isso realmente é melhor do que só perguntar a resposta. Mas, ainda assim, é como se você estivesse escutando alguém te contar como treinou para correr uma maratona. Você pode até entender o que a pessoa fez, mas isso não te torna apto a correr uma maratona. **Você precisa passar pelo processo.**

Além disso, para fins acadêmicos, ferramentas de inteligência artificial são consideradas recursos externos de terceiros. Assim, a utilização de soluções geradas por essas ferramentas como se fossem de autoria própria caracteriza uma forma de cópia indevida, sujeita às mesmas regras e consequências aplicáveis a qualquer outro caso de desonestidade acadêmica.

Por fim, estas orientações têm como objetivo reforçar a importância ética no desenvolvimento dos trabalhos práticos, valorizando o processo de aprendizagem tanto quanto o resultado final.